

Módulo 6: Infraestructura y Deploy

T18: Reverse Proxy, TLS y WAF

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

 *nginx + Let's Encrypt + rate limiting – HTTPS real en producción*

Objetivos

- Configurar nginx como reverse proxy delante de VulnApp
- Obtener un certificado TLS gratuito con Let's Encrypt y certbot
- Implementar rate limiting en nginx para mitigar fuerza bruta
- Configurar un WAF básico con headers y reglas de nginx

¿Por qué un Reverse Proxy?

```
[Internet] → [nginx :443] → [VulnApp :3000]
      |
      ├── TLS termination
      ├── Rate limiting
      ├── Static file caching
      ├── Security headers
      └── WAF rules
```

La app no se expone directamente a Internet. nginx absorbe:

- Ataques DDoS (rate limiting)
- Peticiones malformadas
- Gestión de TLS (la app solo habla HTTP internamente)

Configuración básica de nginx

```
server {  
    listen 443 ssl http2;  
    server_name vulnapp.example.com;  
  
    ssl_certificate      /etc/nginx/ssl/cert.pem;  
    ssl_certificate_key  /etc/nginx/ssl/key.pem;  
    ssl_protocols       TLSv1.2 TLSv1.3;  
    ssl_ciphers          HIGH:!aNULL:!MD5;  
  
    # Security headers  
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;  
    add_header X-Frame-Options DENY always;  
    add_header X-Content-Type-Options nosniff always;  
  
    location / {  
        proxy_pass http://app:3000;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
}
```

Let's Encrypt + certbot

```
# Obtener certificado gratuito
certbot certonly --webroot -w /var/www/certbot \
  -d vulnapp.example.com

# Renovación automática (cron)
0 0 1 * * certbot renew --quiet && nginx -s reload
```

```
# docker-compose.yml con certbot
services:
  nginx:
    image: nginx:alpine
    ports: ["80:80", "443:443"]
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf
      - certbot-data:/etc/letsencrypt
  certbot:
    image: certbot/certbot
    volumes:
      - certbot-data:/etc/letsencrypt
```

Rate Limiting – mitigar fuerza bruta

```
# Definir zona de rate limiting
limit_req_zone $binary_remote_addr zone=login:10m rate=5r/m;
limit_req_zone $binary_remote_addr zone=api:10m rate=30r/m;

server {
    # Login: máximo 5 intentos por minuto
    location /login {
        limit_req zone=login burst=3 nodelay;
        proxy_pass http://app:3000;
    }

    # API general: 30 req/min
    location / {
        limit_req zone=api burst=10;
        proxy_pass http://app:3000;
    }
}
```

WAF básico con nginx

```
# Bloquear user-agents de scanners conocidos
if ($http_user_agent ~* (nikto|sqlmap|nmap|masscan)) {
    return 403;
}

# Bloquear payloads SQLi básicos en la URL
if ($request_uri ~* "(union|select|insert|update|delete|drop).*--") {
    return 403;
}

# Limitar tamaño de body (prevenir uploads gigantes)
client_max_body_size 1m;

# Ocultar versión de nginx
server_tokens off;
```

⚠ Un WAF con reglas regex no sustituye a un WAF completo (ModSecurity, Cloudflare WAF) pero añade una capa útil.

Arquitectura completa de deploy

```
[Internet] → [Cloudflare/CDN] → [nginx :443]
                        |
                        [VulnApp :3000]
                        |
                        [SQLite / PostgreSQL]
```

Capa	Responsabilidad
CDN/Cloudflare	DDoS, caché, WAF global
nginx	TLS, rate limit, proxy, headers
App	Lógica de negocio (sin TLS, sin assets)
BD	Solo accesible desde la app (red interna)

Redirección HTTP → HTTPS

```
# Redirigir TODO el tráfico HTTP a HTTPS
server {
    listen 80;
    server_name vulnapp.example.com;

    # Excepción: challenge de Let's Encrypt
    location /.well-known/acme-challenge/ {
        root /var/www/certbot;
    }

    # Todo lo demás → HTTPS
    location / {
        return 301 https://$host$request_uri;
    }
}
```

⚠ Sin esta redirección, los usuarios que escriben `http://` acceden sin cifrado. Con HSTS + esta redirección, la ventana de exposición es mínima.

OCSP Stapling – verificación de certificados eficiente

```
# El servidor pre-descarga la respuesta OCSP y la envía junto con el cert  
ssl_stapling on;  
ssl_stapling_verify on;  
ssl_trusted_certificate /etc/nginx/ssl/chain.pem;  
resolver 8.8.8.8 8.8.4.4 valid=300s;
```

Sin OCSP Stapling: el navegador consulta a la CA si el cert está revocado (lento, privacy leak).

Con OCSP Stapling: nginx incluye la prueba de validez → faster TLS handshake, mejor privacidad.

Lab T18: nginx + HTTPS con certificado autofirmado

Objetivo: desplegar VulnApp detrás de nginx con HTTPS

Setup: Docker Compose con nginx

1. Añade un servicio `nginx` al `docker-compose.yml` de VulnApp
2. Configura nginx para hacer proxy a VulnApp en el puerto 3000
3. Genera un certificado autofirmado con `openssl req -x509 -newkey rsa:4096`
4. Activa HTTPS en nginx y verifica que `http://` redirige a `https://`

 **Certificados gratuitos en producción:** [Let's Encrypt](#) + certbot